Stela Governance Architecture and Dispatch Strategy: DP-OPS-000030 Analysis

1. Executive Summary and Strategic Imperative

The governance of autonomous code repositories represents one of the most significant challenges in modern software engineering. As artificial intelligence (AI) agents transition from passive assistants to active participants in the development lifecycle—categorized within the Stela framework as "Integrators" and "Contractors"—the mechanisms for controlling their behavior, scope, and context must evolve with rigorous precision. This report provides an exhaustive forensic analysis of the Stela repository's current operational state, specifically targeting the vectors of "drift," "hallucination risks," and "context stagnation" identified in the active work surface.

Acting under the strict authority of the **Policy of Truth (PoT)**, this document serves as the foundational research and rationale for the drafting and execution of **DP-OPS-000030**, a critical governance-focused Dispatch Packet. The primary objective of this intervention is to enforce a **"Pointer-First" Agent Constitution** and execute mandatory housekeeping to restore system equilibrium. Our analysis reveals that the accumulation of historical logs in SoP.md (State of Play) and TASK.md creates a high-risk environment where Large Language Models (LLMs) may "hallucinate reality"—confusing historical actions with current mandates. Furthermore, the repository's context bundles (llms.txt) have stagnated due to script inactivity, and the verification tooling (verify.sh) is flagging false positives on archival storage, impeding the "clean state" requirement essential for secure AI operation.

This report details the architectural principles of "Pointer-First" governance, analyzes the specific friction points in the current HEAD (cc26a4168), and culminates in the fully drafted Dispatch Packet **DP-OPS-000030**. The analysis integrates forensic evidence from the repository dump with broader research on AI control theory, hallucination mitigation, and autonomous agent safety to provide a robust justification for every directive contained within the Dispatch Packet.

2. Theoretical Framework: The Stela Constitutional Model

To construct a valid Dispatch Packet for the Stela ecosystem, one must first deconstruct the immutable laws governing its existence. The repository is not merely a collection of code; it is a governed state where file placement, content, and modification are dictated by the **Policy of Truth (PoT)**. This document serves as the constitution, establishing the "physics" of the

environment in which the AI agents operate.

2.1 The Policy of Truth (PoT) and Filing Doctrine

The PoT.md file [1] serves as the supreme law of the Stela repository. It establishes a "Filing Doctrine" that creates a rigid separation of concerns. This doctrine is not merely organizational; it is a security control designed to prevent agents from confusing executable logic with documentation or temporary work artifacts. The filing doctrine dictates that ops/ is the domain of "Run"—containing binaries, manifests, and automation that enforce the platform boundary. Conversely, docs/ is the domain of "Explain," containing manuals and rationale but never runtime logic. projects/ represents "Work," the payload code that operates under the governance of the platform but must not import from the governance layers. Finally, storage/ is designated as "Trash" or local artifacts, explicitly excluded from the canonical definition of the system.

The rigorous enforcement of this doctrine is critical for preventing "drift," a state where the repository's reality diverges from its governed intent. Drift is a precursor to system failure in autonomous environments. If an agent creates a script in docs/ or places a manual in ops/, it blurs the line between instruction and execution, increasing the probability of erroneous behavior. DP-OPS-000030 must enforce this doctrine by ensuring that the housekeeping tasks requested—specifically the archival of logs and the update of verification scripts—adhere strictly to these jurisdictional boundaries. Any script changes must occur in ops/ or tools/, and any archival of logs must move them to storage/ or a designated archive path that respects the "Trash" designation of the storage directory while preserving auditability.

2.2 The Staffing Protocol and Jurisdictional Authority

The Stela framework defines a precise hierarchy of authority known as the Staffing Protocol. This protocol governs the relationship between human intent and AI execution, establishing clear lines of accountability and capability.

Role	Entity	Jurisdiction	Authority Level
Operator	Human	Total	Owns final decisions, approvals, secrets, commits, pushes, and merges.
Integrator	Lead AI	Governance	Maintains structural integrity, auditing, and generates

			Dispatch Packets.
Contractor	Guest AI	Execution	Executes specific logic tasks and drafts implementation details within scope.

This hierarchy is essential for "AI Control," a concept that seeks to ensure agents produce useful work even when attempting to misbehave or hallucinate.[2] The "Operator" retains the "kill switch" authority—the ability to stop work, reject PRs, and define the base reality of the repository. The "Integrator" acts as the foreman, translating the Operator's intent into structured mandates (Dispatch Packets) that the "Contractor" can execute without ambiguity. The separation of the Integrator (Architecture/Governance) from the Contractor (Coding/Implementation) creates a system of checks and balances. The Integrator drafts the plan (DP-OPS-000030), and the Contractor executes it. If the Contractor deviates, the Integrator or the automated gates (like verify.sh) detect the drift and halt the process.

2.3 Behavioral Logic Standards and Linguistic Precision

The PoT enforces a "Behavioral Logic Standard" characterized by extreme linguistic precision. This standard prohibits contractions (e.g., "don't," "can't") to ensure rhythmic precision for technical auditing. It mandates "Quantitative Reporting," requiring agents to specify exact deviations rather than subjective descriptions. Most critically, it demands "Absolute Literalism." Agents must interpret instructions with total fidelity and seek clarification for any ambiguity.

This linguistic rigor serves as a defense against "prompt injection" and "instruction drift." By forcing agents to communicate in a formal, precise dialect, the system reduces the cognitive load required to parse their outputs and increases the detectability of hallucinations. A hallucinating agent often reverts to conversational, probabilistic language ("I think I fixed it," "It should work now"). A compliant Stela agent reports, "Execution path deviated by 14.7%," or "Verification: NOT RUN." DP-OPS-000030 must reinforce this standard, ensuring that the cleanup of TASK.md and the refreshing of context bundles are reported with the required quantitative precision.

3. The Hallucination Vector: Context Hygiene and Log Pollution

The user's query highlights a critical vulnerability: "METHOD TO PREVENT MODELS READING

IN THE SoP OR TASK LOGS HALLUCINATING REALITY." This identifies a fundamental challenge in Large Language Model (LLM) operations known as "context contamination" or "history bias."

3.1 The Mechanism of Hallucinated Reality

LLMs function by predicting the next token based on the provided context window. When SoP.md (the State of Play ledger) or TASK.md (the active work log) accumulates a large volume of historical data, this data competes for attention within the model's context. If the logs contain records of failed experiments, deprecated methodologies, or past configurations, the model may struggle to distinguish between "what happened in the past" and "what is true today".[3]

For example, if SoP.md contains 50 entries describing a legacy deployment script that was removed in DP-OPS-0020, an agent tasked with deployment in DP-OPS-0030 might retrieve the logic from the log entry and attempt to use it, essentially "hallucinating" that the legacy script still exists. This phenomenon is exacerbated when logs are verbose or contain code snippets. The "signal-to-noise" ratio of the context window degrades, leading to "daydreaming" where the model invents facts based on probabilistic associations found in the historical noise.[4]

Research into "AI Package Hallucination" [4] and database deletion incidents [5] underscores the severity of this risk. In one cited case, an AI agent tasked with maintenance deleted a production database because it interpreted vague instructions through the lens of a "troubleshooting" context rather than a "safety" context. By aggressively pruning historical logs from the active context, we reduce the surface area for such misinterpretations.

3.2 The "Pointer-First" Remediation Strategy

To mitigate this risk, DP-OPS-000030 must implement a "Pointer-First" constitution. This strategy dictates that the active context (the files loaded into the agent's working memory) must contain only the **current** truth and **pointers** to history, never the history itself.

- **Current State:** SoP.md grows indefinitely, creating a massive context burden.
- **Desired State:** SoP.md is truncated to the most recent 30 entries. Older entries are moved to storage/archives/, which is **not** listed in ops/lib/manifests/CONTEXT.md.[1]
- **Mechanism:** The ops/bin/prune script [1] is the designated tool for this hygiene. However, the user reports that verify.sh flags the archives as drift. This creates a deadlock: we cannot prune without causing a verification failure.

The "Pointer-First" principle also applies to the constitution itself. We must ensure that rules are defined in exactly one place (PoT.md) and referenced elsewhere. Duplicating rules (e.g., pasting the "No Contractions" rule into every agent's prompt) creates maintenance drift. If the rule changes in PoT.md but not in the agent prompts, the system enters a schismatic state.

DP-OPS-0030 will audit the repository for such duplications and replace them with pointers to the Single Source of Truth (SSOT).

3.3 Context Stagnation and the llms.txt Failure

The user notes: "llms-small.txt llms-full.txt ARE NOT UPDATING - SCRIPTS ARE NOT RUNNING." This indicates a failure in the continuous integration or manual workflow triggers. llms.txt [1] is the "machine-readable entry point" for the repository. It serves as the map that guides AI agents to the correct documentation and tools. If this map is outdated, agents act on stale intelligence.

The ops/bin/llms script [1] is responsible for generating these bundles. It parses ops/lib/manifests/CONTEXT.md, applies hazard exclusion rules (filtering out JIT resources like agents and skills), and compiles the "Small" and "Full" bundles. The failure of this script to run suggests a gap in the TASK.md closeout procedure. Ideally, every Dispatch Packet should trigger a context refresh as its final action to ensure the next session starts with an accurate map. DP-OPS-000030 must enforce this by adding an explicit execution step for ops/bin/llms and verifying the timestamps of the output files.

4. Forensic Audit of Operational Drift

To prepare DP-OPS-000030, we must perform a forensic audit of the specific friction points reported by the Operator. This involves analyzing the code within the provided dump to understand why the system is drifting.

4.1 The verify.sh Storage Warning

The Operator reports: "verify.sh warning about storage/archives is pre-existing storage drift."

Analysis of tools/verify.sh [1]: The script contains a "Storage Hygiene Check" section:

Bash

```
# Drift check: warn on unexpected clutter in storage/
# Allowed: README.md,.gitignore, handoff, dumps, dp, _scratch
for item in storage/*; do
  name="$(basename "$item")"
  case "$name" in
    README.md|.gitignore|handoff|dumps|dp|_scratch)
      ;;
    *)
```

```
    warn "Storage drift: unexpected item 'storage/$name'. Keep storage/ clean."
    ;;
  esac
done
```

The script explicitly allows handoff, dumps, dp, and _scratch, but it does **not** allow archives. When ops/bin/prune moves old SoP entries to storage/archives/, verify.sh correctly identifies this new directory as "unexpected clutter" based on its hardcoded allowlist.

Remediation: We must modify tools/verify.sh to add archives to the case statement. This validates the archival strategy as a sanctioned operational pattern rather than accidental drift.

4.2 The TASK.md Receipt Failure

The user states: "LAST DP RECEIVED NO RECEIPT BUNDLE - NO open/porcelain/metadata/results."

Analysis of TASK.md [1]: The TASK.md file contains a strict "Receipt (Required)" section:

> "MANDATE: No receipt/proof bundle = AUTOMATIC DISAPPROVAL. Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition."

The failure to produce this receipt in the previous cycle represents a governance breach. It implies the Contractor or Integrator skipped the verification steps or failed to execute the final dump. This breaks the chain of custody. Without the "OPEN" artifact, we cannot prove the state of the repo when work began. Without the "RESULTS" artifact, we cannot prove what changed.

Remediation: DP-OPS-000030 must set a precedent by strictly enforcing its own receipt generation. It must also include a "Cleanup" step in its Execution Plan to generate a retrospective log entry or "schema reset" for TASK.md to formally close the broken loop of the previous DP, ensuring the ledger acknowledges the missing receipt rather than ignoring it.

4.3 The "Pointer-First" Constitution Gap

The user requests: "Pointer-first agent constitution for Stela (no duplicated canon text)."

Analysis of Agent Files [1]: Files like docs/library/agents/R-AGENT-01.md currently contain sections like:

Role

Provide architecture guidance...

Specialization

Architecture planning...

Pointers

- Constitution: PoT.md
  ...
  However, older versions or other documentation files might contain copies of the "Behavioral Logic Standard." A forensic scan (using grep) is required to ensure no file other than PoT.md defines the core rules. If docs/GOVERNANCE.md or .github/copilot-instructions.md repeats the rules verbatim, they must be refactored to say "Refer to PoT.md Section 2."

This "Pointer-First" approach reduces the token count of the context (saving cost and memory) and eliminates the risk of conflicting instructions. It aligns with the "Reuse-First Discipline" of the PoT.[1]

5. Architectural Remediation Strategy

Based on the theoretical framework and forensic audit, the remediation strategy for DP-OPS-000030 is defined as follows:

5.1 The Archival Pipeline

We will formalize the storage/archives/ directory as the designated "Cold Storage" for the repository.

- **Action:** Create storage/archives/.gitkeep to ensure the directory structure persists (if not already present).
- **Action:** Update tools/verify.sh to whitelist archives in the storage drift check.
- **Action:** Execute ops/bin/prune to move SoP.md entries older than 30 events to this cold storage.
- **Action:** Manually scrub TASK.md of previous thread logs, moving them to a TASK-archive.md in storage if necessary, or simply deleting them if they are captured in previous RESULTS bundles (as per the "Trash" doctrine of storage).

5.2 The Context Refresh Loop

We will enforce a "Liveness Check" for context bundles.

- **Action:** The Execution Plan for DP-OPS-000030 will include a mandatory step to run ops/bin/llms.
- **Action:** We will verify that llms-small.txt and llms-full.txt are updated. This ensures that the "Entry Points" defined in PoT Section 4.4 are functional.

5.3 The Governance Refactor (Pointer-First)

We will audit and refactor the core governance files to ensure SSOT.

- **Action:** Review docs/GOVERNANCE.md and docs/QUICKSTART.md. If they recite the "Contraction Prohibition" or "Staffing Protocol" in full, replace those sections with a link: "See PoT.md Section X."
- **Action:** Ensure TASK.md itself does not duplicate PoT definitions but references them.

5.4 Linting Enforcement

The user emphasizes: "STYLE/LLMS markdownlint THESE AND OTHERS SHOULD BE RUNNING WITH EVERY DP CYCLE."

- **Action:** DP-OPS-000030 will include tools/lint/style.sh (markdownlint wrapper) and tools/lint/llms.sh (context bundle verifier) in its "Verification" section.
- **Action:** This reinforces the "Green Gates Only" rule of docs/GOVERNANCE.md.[1]

6. Dispatch Packet Construction: DP-OPS-000030

The following section presents the fully constructed Dispatch Packet. It adheres strictly to the TASK.md schema, utilizing the specific headings, order, and closing blocks required by the Policy of Truth. It is designed to be the "Supreme Focus" requested by the Operator, restoring order and enabling forward momentum.

6.1 DP Specification and Rationale

Section	Content	Rationale
Freshness Gate	main / cc26a4168	Anchors work to the verified state from the provided OPEN prompt.
Context Load	Standard Canon + verify.sh, prune, llms	Loads the specific tools that require modification or execution.
Scope	Platform / Governance	Strictly limits work to ops/, docs/, and tools/. projects/ is forbidden.
Execution Plan	Prune logs, Fix verify, Refresh LLMs	The sequential logic to resolve the identified

		friction points.
Receipt	Strict Bundle	Enforces the "Forward Momentum/Accountability" requirement.

DP-OPS-000030: Pointer-First Agent Constitution & Hygiene

1. Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/governance-refactor-2026-02-06

Base HEAD: cc26a4168

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

2. Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- PoT, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - tools/verify.sh

    - ops/bin/llms
    - ops/bin/prune

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN).

DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3. Scope and Safety

- **Objective:** Enforce Pointer-First constitution (no duplicated canon), resolve verify.sh storage drift, and force-refresh stagnant context bundles.
- **Non-Goals (optional, drift-killer):**
    - No changes to projects/ logic.
    - No changes to PoT.md core axioms (only reference handling).

Target Files allowlist (hard gate)

- TASK.md
- SoP.md
- tools/verify.sh
- storage/archives/.gitkeep (new)
- llms-small.txt
- llms-full.txt

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
    - No edits to .github/
    - No edits to projects/ unless explicitly authorized
    - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
    - Workers do not create/switch branches.
    - Workers do not stage/commit/merge/push.
    - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **PoT.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
    - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
    - STOP if required DP inputs are missing (do not guess).
    - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

4. Execution Plan (A-E Canon)

4.1 State

- **Context:** The repository is suffering from "Hallucination of Reality" due to historical logs cluttering TASK.md and SoP.md. Context bundles (llms.txt) are stale. verify.sh incorrectly flags archival storage as drift.
- **Drift:** verify.sh fails on storage/archives. llms.txt timestamp is lagging.
- **Desired State:** TASK.md is focused (no old logs). SoP.md is pruned. verify.sh accepts archives. Context bundles are fresh. Pointer-first doctrine is respected.

4.2 Request

Numbered tasks (concrete, no vibes):

1. **Fix Verification:** Edit tools/verify.sh. Update the "Drift check" loop to explicitly allow archives inside storage/. Ensure storage/archives/ exists with a .gitkeep.
2. **Prune SoP:** Run ./ops/bin/prune. Confirm old entries are moved to storage/archives/.
3. **Refine TASK.md:**
   - Remove all historical "Work Log" entries from TASK.md.
   - Insert a static warning under the Work Log header: > **NOTICE:** This log is for the *active* DP only. Historical logs are archived in 'storage/archives/'. Do not rely on this log for long-term history.
   - Reset the "Current DP" section to DP-OPS-000030.
4. **Refresh Context Bundles:**
   - Run ./ops/bin/llms.
   - Verify llms-small.txt and llms-full.txt have updated timestamps.
5. **Update SoP:** Add an entry for this DP (Governance Refactor & Hygiene).

Required Content:

- Ensure tools/verify.sh drift check modification uses precise glob matching or case statement to allow archives.

4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
  - Updated tools/verify.sh to allow storage/archives (fixing storage drift warning).
  - Pruned SoP.md and archived historical logs.
  - Cleared TASK.md work log to prevent context hallucination.
  - Refreshed llms-small.txt and llms-full.txt.

4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
  - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
  - Prefer: bash tools/lint/context.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when PoT or canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint/truth.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution. Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
  - Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-000030-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-work-governance-refactor-2026-02-06-*.txt (required)
- storage/handoff/OPEN-work-governance-refactor-2026-02-06-*.txt (required)
- storage/dumps/dump-platform-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)

- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

5. Closeout (Mandatory)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-000030-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above). (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording; copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

4.1 Thread Transition (Reset / Archive Rule)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **2. Active Context** and **3. Current Dispatch Packet** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---
   - THREAD START: DP-OPS-

(If the Work Log gets too long, it’s acceptable to keep only the most recent thread’s entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

6. Work Log (Timestamped Continuity)

Rule: This is not a transcript. It’s the durable breadcrumb trail.

Expectation: Append a new timestamped entry on every DP closeout (no exceptions).

Each entry ends with: NEXT: <one single action>.

- ** — DP-OPS-: [short note]. Verification:. Blockers: [none/...]. NEXT: [single action]

---

7. Conclusions and Operational Roadmap

The construction of DP-OPS-000030 represents a pivotal moment in the Stela governance maturity cycle. By addressing the "Hallucination of Reality" through strict log hygiene and "Pointer-First" references, we effectively close a significant vector of agent misalignment. The remediation of the verify.sh drift warning eliminates a persistent "Broken Windows" effect, where false positive warnings desensitize agents to actual failures.

However, the success of this Dispatch Packet relies on the unwavering "Absolute Literalism" of the Contractor agent executing it. The risk remains that a Contractor might attempt to "improve" the TASK.md format rather than strictly adhering to the schema, or fail to produce the complete receipt bundle due to process fatigue. The "Mandatory Closing Block" and strict "Receipt" requirements in Section 4.5 of the DP are designed to counter these behavioral risks.

Future operational cycles must continue to enforce the "Reuse-First Discipline." As the ops/ directory matures, the need for ad-hoc scripting should diminish. Integrators must be vigilant in detecting "Logic Drift," where agents begin to invent new patterns instead of leveraging existing ops/lib/ resources. The successful execution of DP-OPS-000030 will serve as the validation of this rigorous, constitution-first approach to AI-assisted software development.

Works cited

1. dump-full-main-cc26a4168.txt
2. AI Control: How to Make Use of Misbehaving AI Agents | Center for Security and Emerging Technology, accessed February 6, 2026, https://cset.georgetown.edu/article/ai-control-how-to-make-use-of-misbehaving-ai-agents/
3. Stop LLM Hallucinations: Reduce Errors by 60–80% - Master of Code, accessed February 6, 2026, https://masterofcode.com/blog/hallucinations-in-llms-what-you-need-to-know-before-integration
4. When LLMs day dream: Hallucinations and how to prevent them - Red Hat, accessed February 6, 2026, https://www.redhat.com/en/blog/when-llms-day-dream-hallucinations-how-prevent-them
5. 3 safeguards every enterprise needs to prevent AI agent misalignment - Box Blog, accessed February 6, 2026, https://blog.box.com/3-safeguards-every-enterprise-needs-prevent-ai-agent-misalignment